

Sumario

GS - Testing, Compliance & Quality Assurance	1
Mission Clear - Monitoramento de Detritos Espaciais e Simulação de Missões LEO Seguras . . .	1
1. Identificação da entrega	1
2. Breve descritivo do projeto	2
3. Visão da arquitetura da solução no padrão TOGAF	2
4. Arquitetura de negócio no padrão TOGAF	5
5. Arquitetura de sistema no padrão TOGAF	6
6. Arquitetura de tecnologia no padrão TOGAF	8
7. Aplicação em Testing, Compliance & Quality Assurance	9
8. Compliance e qualidade	12
9. Relação com ArchiMate	13
10. Checklist final da entrega	13
11. Pendências manuais antes do envio	13

GS - Testing, Compliance & Quality Assurance

Mission Clear - Monitoramento de Detritos Espaciais e Simulação de Missões LEO Seguras

Projeto acadêmico: FIAP - Global Solution 2026

Matéria: Testing, Compliance & Quality Assurance

Tema geral da GS: Viagens ao Espaço

Projeto aplicado: Mission Clear

1. Identificação da entrega

Integrante	RM
Gustavo Bezerra Assunção	553076
Jó Sales	552679
Miguel Garcez de Carvalho	553768
Vinicius Souza e Silva	552781
Edson Leonardo	553737

Item	Informação
Repositório backend	https://github.com/FIAP-BitsNBytes/Fiap-3ESPR-GS-C
Repositório mobile	https://github.com/FIAP-BitsNBytes/Fiap-3ESPR-GS-Mobile
Link do vídeo pitch	INSERIR LINK DO YOUTUBE NÃO LISTADO AQUI
Arquivo ArchiMate	INSERIR/ANEXAR ARQUIVO .archimate
PDF do diagrama	INSERIR/ANEXAR PDF EXPORTADO DO ARCHI EM ALTA RESOLUÇÃO

Observação: conforme a regra da entrega, este documento deve concentrar a identificação da equipe, o link do vídeo pitch e a documentação da arquitetura. O diagrama oficial deve ser criado no Archi/ArchiMate, salvo como .archimate e exportado em PDF de alta resolução.

2. Breve descritivo do projeto

O **Mission Clear** é uma solução voltada à segurança de missões espaciais em órbita baixa terrestre, conhecida como **LEO - Low Earth Orbit**. O projeto parte do problema do crescimento de detritos espaciais, como fragmentos de satélites, partes de foguetes e objetos orbitais que podem cruzar trajetórias de missões em alta velocidade.

A proposta do sistema é apoiar a análise de risco antes e durante uma simulação de missão. Para isso, a solução utiliza dados reais de detritos espaciais fornecidos pela **CelesTrak**, propaga as órbitas por meio da biblioteca **SGP4** e calcula janelas de lançamento com menor risco de conjunção.

A solução é composta por dois blocos principais:

Bloco	Descrição
Backend	API em ASP.NET Core/.NET responsável por ingestão de dados orbitais, autenticação, cálculo de risco, geração de janelas de lançamento, simulação de missão e histórico.
Mobile	Aplicativo React Native/Expo responsável pela visualização do dashboard, exploração de detritos, favoritos, histórico, simulação de missão e consumo dos dados da API.

Em alto nível, o fluxo da solução é:

CelesTrak TLEs -> SGP4 Propagation -> Conjunction Detection -> Safe Launch Windows -> Mission

O projeto se relaciona principalmente aos ODS 9, 11 e 13, ao propor uma solução tecnológica associada à infraestrutura espacial, sustentabilidade da infraestrutura orbital e apoio à observação climática dependente de satélites.

3. Visão da arquitetura da solução no padrão TOGAF

3.1 Stakeholders do projeto

Stakeholder	Interesse no projeto
Operador de missão	Simular uma missão orbital e identificar janelas de lançamento com menor risco.
Pesquisador	Consultar dados orbitais, detritos, estatísticas e alertas de conjunção.
Administrador	Gerenciar permissões, usuários e acesso a recursos restritos.
Equipe de desenvolvimento	Implementar API, app mobile, testes, documentação e integração entre módulos.
Avaliadores FIAP	Verificar aderência técnica, arquitetura TOGAF/ArchiMate, QA e documentação.
CelesTrak	Fonte externa de dados orbitais utilizados pela solução.
Usuário final do app	Visualizar informações orbitais e simular cenários de missão de forma acessível.

3.2 Drivers do projeto

Driver	Descrição
Segurança orbital	Reduzir a exposição de missões simuladas a riscos de aproximação com detritos espaciais.
Uso de dados reais	Consumir TLEs da CelesTrak em vez de depender apenas de dados fictícios.
Simulação e decisão	Apoiar a escolha de janelas de lançamento com menor risco.
Integração backend/mobile	Garantir que a análise feita na API seja consumida corretamente pelo app.
Qualidade de software	Validar API, autenticação, serviços, mobile, erros e streaming SSE.
Observabilidade	Permitir acompanhamento do estado da aplicação, logs, traces e health checks.
Rastreabilidade	Relacionar requisitos, componentes, testes, riscos e evidências da entrega.

3.3 Objetivos e metas

Objetivo	Meta associada
Disponibilizar dados orbitais	Carregar dados da CelesTrak e expor informações por endpoints da API.
Propagar órbitas	Usar biblioteca SGP4 para calcular posição e velocidade dos objetos rastreados.
Detectar conjunções	Identificar detritos que se aproximam da rota simulada dentro do limite de risco definido no projeto.
Gerar janelas seguras	Dividir o período em slots e destacar janelas com menor risk_score.
Simular missões	Permitir simulação estática e dinâmica com eventos SSE.
Proteger dados de acesso	Usar JWT, refresh token, BCrypt e armazenamento seguro no mobile.
Registrar histórico	Salvar e consultar missões simuladas por usuários autenticados.
Documentar arquitetura	Representar negócio, aplicação, dados e tecnologia no Archi/ArchiMate.

3.4 Requisitos funcionais

Código	Requisito funcional
RF01	Consultar o status da API e o estado de carregamento dos dados orbitais.
RF02	Listar destinos de missão disponíveis.
RF03	Listar detritos espaciais e objetos orbitais propagados.
RF04	Exibir estatísticas de detritos por tipo ou faixa orbital.
RF05	Consultar detalhes de um objeto orbital específico.
RF06	Calcular janelas de lançamento.
RF07	Retornar as melhores janelas de lançamento por menor risco.
RF08	Executar simulação estática de missão.

Código	Requisito funcional
RF09	Criar sessão dinâmica de simulação de missão.
RF10	Transmitir eventos de missão via SSE.
RF11	Cadastrar, autenticar, renovar e encerrar sessão de usuários.
RF12	Consultar e atualizar perfil do usuário autenticado.
RF13	Consultar histórico e estatísticas de missões.
RF14	Exibir dashboard mobile com status, estatísticas e alertas.
RF15	Persistir favoritos e preferências no app mobile.

3.5 Requisitos não funcionais

Código	Requisito não funcional
RNF01	A API deve usar JSON padronizado em snake_case.
RNF02	A autenticação deve usar JWT Bearer e refresh token.
RNF03	Senhas devem ser armazenadas com hash seguro usando BCrypt.
RNF04	A API deve possuir tratamento global de exceções.
RNF05	O backend deve possuir separação entre Controllers, Services, Repositories, DTOs e Entities.
RNF06	Os serviços principais devem ser testáveis por testes unitários.
RNF07	O app mobile deve armazenar tokens de forma segura usando SecureStore.
RNF08	Preferências e favoritos do app devem ser persistidos localmente.
RNF09	O sistema deve possuir observabilidade com OpenTelemetry e Aspire Dashboard.
RNF10	A integração mobile/backend deve funcionar via REST e SSE.

3.6 Constraints do projeto

Restrição	Impacto na arquitetura
Backend em .NET / ASP.NET Core	Define runtime, Controllers, Services, DI e integração com Aspire.
Banco MySQL 8	Define persistência relacional, EF Core e migrations.
React Native + Expo	Define a camada mobile e a estratégia de armazenamento local.
Uso de CelesTrak	Exige integração externa HTTP para dados orbitais.
Uso de SGP4 por biblioteca	Evita reimplementação manual da propagação orbital.
JWT + Refresh Token	Define autenticação stateless com renovação de sessão.
SSE	Define canal de stream para simulações dinâmicas.

Restrição	Impacto na arquitetura
Entrega acadêmica no Teams	Exige documentação única, vídeo pitch, arquivo .archimate e PDF do diagrama.

4. Arquitetura de negócio no padrão TOGAF

4.1 Processo de negócio principal

Processo: Simular uma missão orbital segura.

O processo de negócio representa o fluxo pelo qual um operador ou pesquisador utiliza o Mission Clear para analisar riscos e escolher uma janela de lançamento mais segura.

4.2 Tarefas do processo

Ordem	Tarefa
1	Usuário acessa o app Mission Clear.
2	App consulta o status da API.
3	Backend carrega ou disponibiliza dados orbitais da CelesTrak.
4	Usuário escolhe destino de missão, como ISS, LEO genérica ou SSO.
5	Sistema calcula janelas de lançamento em slots.
6	Sistema analisa risco de aproximação com detritos.
7	App apresenta janelas recomendadas e alertas.
8	Usuário inicia simulação estática ou dinâmica.
9	Em simulação dinâmica, a API transmite eventos SSE.
10	Sistema finaliza a simulação e calcula o resultado.
11	Se autenticado, o usuário pode consultar a missão no histórico.

4.3 Localidade de operação

Localidade	Operação realizada
Dispositivo mobile	Login, dashboard, favoritos, consulta de detritos e simulação.
Backend API	Processamento orbital, autenticação, cálculo de risco e endpoints REST/SSE.
Banco MySQL	Persistência de usuários, refresh tokens e missões.
CelesTrak	Fornecimento externo de dados orbitais.
Aspire Dashboard	Monitoramento de logs, traces e health checks.
Microsoft Teams	Ambiente de envio oficial da entrega acadêmica.

4.4 Operações realizadas

Operação	Descrição
Consulta orbital Cálculo de risco	Listagem de detritos e estatísticas orbitais. Avaliação de aproximação entre missão simulada e detritos.
Simulação	Execução de cenário de missão com resultado imediato ou streaming.
Autenticação Persistência	Controle de acesso por JWT, refresh token e roles. Registro de usuários, tokens, histórico e preferências.
Observabilidade	Acompanhamento técnico do estado da solução.

4.5 Papéis envolvidos

Papel	Responsabilidade
Operador de missão	Seleciona destino, consulta janelas e acompanha simulação.
Pesquisador	Analisa detritos, estatísticas e alertas.
Administrador	Acessa recursos restritos e gerencia permissões.
Sistema Mission Clear	Processa dados, calcula risco e retorna resultados.
Fonte externa CelesTrak	Disponibiliza dados TLE para consumo do backend.

5. Arquitetura de sistema no padrão TOGAF

5.1 Camadas de software

Camada	Componentes
Apresentação	App mobile React Native/Expo e interface web MVC do backend.
Navegação mobile	React Navigation com Stack e Bottom Tabs.
Estado mobile	Context API para autenticação, favoritos e tema.
Serviços mobile	Axios, interceptors JWT, tratamento de erro e SSE.
API backend	ASP.NET Core Controllers.
Negócio backend	AuthService, OrbitalEngineService, LaunchWindowService, ConjunctionDetector e MissionSessionService.
Dados backend	Repositories, EF Core, Entities, DTOs e MySQL.
Observabilidade	OpenTelemetry, health checks e Aspire Dashboard.

5.2 Componentes do backend

Componente	Responsabilidade
MissionClear.Api	API REST e motor orbital principal.
MissionClear.AppHost	Orquestração local com .NET Aspire.
MissionClear.ServiceDefaults	Configurações comuns de observabilidade e health checks.

Componente	Responsabilidade
MissionClear.Web	Interface web MVC com autenticação por cookie e roles.
MissionClear.Tests	Projeto de testes automatizados com xUnit, FluentAssertions e Moq.
TleIngestionService	Consome dados da CelesTrak.
OrbitalCache	Armazena dados orbitais em memória.
OrbitalEngineService	Propaga objetos orbitais via SGP4.
ConjunctionDetector	Detecta aproximações perigosas.
LaunchWindowService	Calcula e classifica janelas de lançamento.
AuthController/AuthService/JwtService	Gerencia cadastro, login, refresh token e logout.
GlobalExceptionHandler	Padroniza respostas de erro.

5.3 Componentes do mobile

Componente	Responsabilidade
Screens	Telas de dashboard, busca, detalhes, simulação, histórico e preferências.
Navigation	Organização das rotas e fluxo de navegação.
Services	Consumo da API por Axios e conexão SSE.
Hooks	Reutilização de lógica de interface e dados.
Contexts	Estados globais de autenticação, favoritos e tema.
Storage	Persistência local com AsyncStorage e SecureStore.
Theme	Tokens visuais, cores, tipografia e suporte a dark/light mode.
Types	Tipagens TypeScript para API e navegação.

5.4 Componentes de dados

Dado	Uso
User	Representa o usuário autenticado.
RefreshToken	Permite renovação da sessão.
Mission	Representa uma missão simulada e salva no histórico.
MissionSession	Representa uma sessão dinâmica de simulação.
OrbitalObject	Representa detrito ou objeto orbital.
LaunchWindow	Representa janela de lançamento e seu risco.
ConjunctionAlert	Representa alerta de aproximação.
Destination	Representa destino orbital da missão.
RiskScore	Representa pontuação de risco calculada.

5.5 Relação entre componentes de software e dados

Componente	Dados acessados
AuthService	User, RefreshToken
MissionService	Mission, MissionSession, RiskScore
LaunchWindowService	Destination, LaunchWindow, RiskScore
ConjunctionDetector	OrbitalObject, ConjunctionAlert
OrbitalEngineService	OrbitalObject, dados TLE carregados em cache
Mobile App	Dados de dashboard, missão, detritos, favoritos, histórico e sessão

Componente	Dados acessados
MySQL	User, RefreshToken, Mission e demais dados persistentes da aplicação

6. Arquitetura de tecnologia no padrão TOGAF

6.1 Equipamentos e ambientes

Recurso	Papel na solução
Smartphone ou emulador	Execução do app React Native/Expo.
Máquina de desenvolvimento	Execução local do backend, mobile, banco e testes.
Servidor backend	Execução da API ASP.NET Core.
Servidor MySQL	Persistência relacional.
Internet	Comunicação com CelesTrak e publicação do vídeo pitch.
GitHub	Versionamento dos repositórios.
Microsoft Teams	Envio oficial da entrega.

6.2 Servidores e infraestrutura

Elemento	Descrição
.NET Runtime	Executa a API e os serviços associados.
.NET Aspire	Orquestra a execução local e o dashboard de observabilidade.
MySQL 8	Banco de dados relacional da solução.
Expo	Ambiente de execução e teste do app mobile.

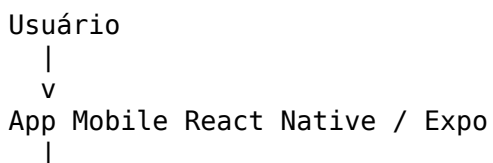
6.3 Banco de dados

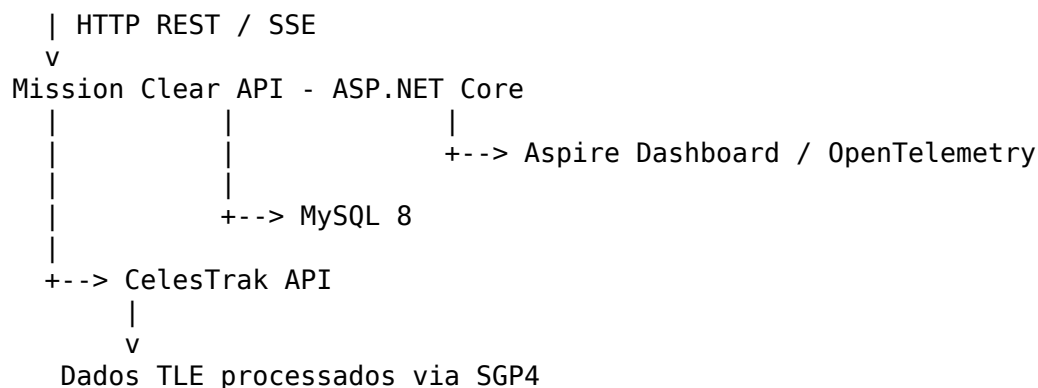
O banco de dados utilizado é o **MySQL 8**, acessado pelo backend por meio do **Entity Framework Core** com provider Pomelo. Ele armazena informações relacionadas a usuários, refresh tokens e missões simuladas.

6.4 Integrações externas

Integração	Uso
CelesTrak	Fonte externa de TLEs e dados orbitais.
SGP4 Library	Biblioteca usada para propagação orbital.
YouTube	Hospedagem do vídeo pitch como não listado.
GitHub	Disponibilização dos repositórios backend e mobile.

6.5 Topologia de rede





6.6 Padrões de conectividade

Comunicação	Padrão
Mobile -> API	HTTP/REST com JSON.
Mobile -> API Streaming	Server-Sent Events.
API -> MySQL	EF Core / conexão relacional.
API -> CelesTrak	HTTP GET.
API -> Aspire Dashboard	OpenTelemetry, logs e traces.

7. Aplicação em Testing, Compliance & Quality Assurance

7.1 Estratégia de testes

A estratégia de QA do Mission Clear deve validar o comportamento da API, a integração com dados orbitais, a segurança da autenticação, o consumo pelo app mobile, o canal SSE e a robustez do tratamento de erros.

O backend já indica uso de **xUnit**, **FluentAssertions** e **Moq** para testes automatizados, com foco em cobertura dos serviços. O app mobile deve ser validado por testes funcionais manuais e, se houver tempo, por testes automatizados de componentes e integração com a API.

7.2 Testes unitários

Alvo	Objetivo
AuthService/JwtService	Validar login, geração de token e regras de sessão.
LaunchWindowService ConjunctionDetector	Validar geração de slots e classificação por risco. Validar identificação de aproximações e níveis de risco.
OrbitalEngineService	Validar chamadas à biblioteca SGP4 e retorno de dados propagados.
MissionService/MissionSessionService	Validar criação, execução e finalização de simulação.
Repositories	Validar regras de acesso a dados de forma isolada.

7.3 Testes de integração

Integração	Validação esperada
API + MySQL	Persistência e consulta de usuários, tokens e missões.
API + CelesTrak	Carregamento de dados orbitais quando disponível.
API + Serviços internos	Injeção de dependência e fluxo entre Controllers, Services e Repositories.
Mobile + API	Consumo correto dos endpoints e tratamento de erro.
API + SSE	Stream de eventos de missão funcionando até o encerramento.

7.4 Testes de API

Endpoint	O que validar
GET /api/status	Status de carregamento e disponibilidade da API.
GET /api/destinations	Destinos disponíveis.
GET /api/debris	Lista de detritos propagados.
GET /api/debris/stats	Estatísticas de detritos.
GET /api/launch-windows	Janelas geradas.
GET /api/launch-windows/best	Melhores janelas.
POST /api/mission/simulate	Simulação estática.
POST /api/mission/session	Criação de sessão dinâmica.
GET /api/mission/session/{id}/stream	Funcionamento do stream SSE.
POST /api/auth/register	Cadastro de usuário.
POST /api/auth/login	Login e retorno de tokens.
POST /api/auth/refresh	Renovação de access token.
POST /api/auth/logout	Invalidação de sessão.
GET /api/users/me	Perfil autenticado.
GET /api/missions	Histórico de missões.
GET /api/dashboard/summary	Resumo do dashboard.
GET /api/dashboard/alerts	Alertas ativos.

7.5 Testes de autenticação e autorização

Cenário	Resultado esperado
Login válido	Retorno de access token e refresh token.
Login inválido	Erro padronizado, sem exposição de dados sensíveis.
Refresh token válido	Novo access token emitido.
Refresh token expirado/inválido	Sessão recusada.
Acesso sem token a rota protegida	Bloqueio de acesso.
Usuário Researcher	Acesso apenas aos recursos permitidos.
Usuário Administrator	Acesso aos recursos administrativos previstos.

7.6 Testes mobile

Módulo	Validação
Login/Registro	Fluxo de autenticação e persistência da sessão.
Dashboard	Exibição de status, gráficos e alertas.
Detritos	Busca, filtros e detalhes técnicos.

Módulo	Validação
Favoritos	Persistência local dos objetos favoritos.
Simulação	Envio de parâmetros e exibição de resultado.
Histórico	Registro e visualização de missões simuladas.
Tema	Alternância entre dark/light mode.
Erro de conexão	Mensagem adequada quando a API estiver indisponível.

7.7 Testes de SSE

Evento	Validação
Criação de sessão	ID de sessão retornado corretamente.
heartbeat	Conexão mantida ativa.
debris_update	Posições de detritos próximos atualizadas.
conjunction_alert	Alerta emitido quando houver risco.
session_complete	Encerramento correto da simulação.
Falha de conexão	App trata o erro sem travar.

7.8 Testes de tratamento de erro

Cenário	Resultado esperado
Erro de validação	Retorno claro com mensagem padronizada.
Erro de autenticação	Retorno 401 ou equivalente.
Erro de autorização	Retorno 403 ou equivalente.
Recurso inexistente	Retorno 404 ou equivalente.
Exceção de domínio	Envelope { error, message, timestamp }.
Erro inesperado	Retorno controlado sem vaziar stack trace.

7.9 Testes de regressão

A cada alteração relevante, devem ser repetidos os testes dos fluxos principais:

1. consultar status da API;
2. listar detritos;
3. calcular janelas de lançamento;
4. simular missão;
5. autenticar usuário;
6. acessar histórico;
7. validar SSE;
8. validar app mobile consumindo a API.

7.10 Critérios de aceite

Critério	Situação esperada
API executa sem erro crítico	Obrigatório.
Endpoints principais respondem	Obrigatório.
Autenticação funciona	Obrigatório.
Mobile consome API	Obrigatório.
Simulação estática funciona	Obrigatório.
Stream SSE funciona ou está documentado com evidência	Obrigatório para demonstração completa.
Diagrama ArchiMate foi criado no Archi	Obrigatório para a matéria.

Critério	Situação esperada
PDF do diagrama foi exportado Documento contém nomes, RMs e link do vídeo	Obrigatório para visualização completa. Obrigatório.

8. Compliance e qualidade

8.1 Segurança

O projeto utiliza autenticação baseada em JWT Bearer, refresh token, roles e hash de senha com BCrypt. A qualidade da segurança deve ser validada por testes de login, renovação de token, bloqueio de acesso indevido e não exposição de dados sensíveis em erros ou logs.

8.2 Privacidade e proteção de tokens

No app mobile, os tokens devem ser armazenados de forma segura utilizando SecureStore. Preferências e favoritos podem utilizar AsyncStorage, pois não possuem o mesmo grau de sensibilidade dos tokens de autenticação.

8.3 Rastreabilidade

O histórico de missões permite rastrear simulações executadas por usuários autenticados. A matriz de requisitos também relaciona funcionalidades, componentes, evidências e critérios de aceite.

8.4 Observabilidade

O backend possui suporte a OpenTelemetry e Aspire Dashboard, permitindo acompanhamento de logs, traces, health checks e comportamento dos serviços durante execução e testes.

8.5 Manutenibilidade

A separação entre Controllers, Services, Repositories, DTOs e Entities reduz acoplamento e facilita manutenção. No mobile, a organização por screens, services, hooks, contexts, storage, theme e types melhora a clareza estrutural.

8.6 Padronização de API

A API adota JSON em snake_case, DTOs e middleware global de exceções com envelope padronizado. Isso melhora previsibilidade, testes de contrato e integração com o app mobile.

8.7 Evidências esperadas

Evidência	Finalidade
Print da API rodando	Comprovar execução do backend.
Print do Aspire Dashboard	Comprovar observabilidade.
Print do app mobile	Comprovar interface e consumo da API.
Print dos testes	Comprovar QA.
Print ou PDF do diagrama no Archi	Comprovar arquitetura ArchiMate.
Arquivo .archimate	Comprovar uso da ferramenta Archi.
Link do vídeo pitch	Comprovar apresentação da GS.

9. Relação com ArchiMate

O diagrama oficial no Archi deve representar quatro camadas principais:

Camada	Elementos esperados
Negócio	Operador de missão, pesquisador, administrador, processo de simulação e resultado esperado.
Aplicação	Mobile App, API, Auth Service, Orbital Engine Service, TLE Ingestion, Launch Window Service, Conjunction Detector e Mission Session Service.
Dados	User, RefreshToken, Mission, MissionSession, OrbitalObject, LaunchWindow, ConjunctionAlert, Destination e RiskScore.
Tecnologia	Smartphone/emulador, servidor .NET, MySQL, Internet, CelesTrak API, SGP4 Library, Aspire Dashboard, GitHub e YouTube.

Fluxo principal a ser representado:

Operador de Missão -> Mission Clear Mobile App -> Mission Clear API -> Serviços de Backend ->

O arquivo ARCHIMATE_ROTUIRO.md contém o passo a passo detalhado para montar o diagrama no Archi.

10. Checklist final da entrega

Item	Status	Observação
Documento único criado	Concluído	Este arquivo consolida a entrega escrita.
Nomes e RMs inseridos	Concluído	Seção 1.
Links dos repositórios inseridos	Concluído	Seção 1.
Link do vídeo pitch inserido	Pendente	Inserir após publicar no YouTube como não listado.
Arquitetura TOGAF documentada	Concluído	Seções 3 a 6.
Estratégia de QA documentada	Concluído	Seção 7.
Compliance documentado	Concluído	Seção 8.
Roteiro ArchiMate criado	Concluído	Arquivo ARCHIMATE_ROTUIRO.md.
Arquivo .archimate gerado	Pendente	Deve ser feito no Archi.
Diagrama exportado em PDF	Pendente	Exportar do Archi em alta resolução.
Vídeo pitch gravado	Pendente	Até 3 minutos.
Revisão final realizada	Pendente	Conferir antes do envio no Teams.

11. Pendências manuais antes do envio

1. Montar o diagrama no Archi seguindo o roteiro.
2. Salvar o arquivo .archimate.
3. Exportar o diagrama em PDF de alta resolução.
4. Gravar vídeo pitch de até 3 minutos.
5. Publicar o vídeo no YouTube como não listado.

6. Inserir o link do vídeo na Seção 1 deste documento.
7. Revisar nomes, RMs e anexos antes de enviar no Teams.